

Hibernate Architecture

Chapter 04 - JPA and Hibernate | Sub-Chapter 04.02

Beginner-Friendly Deep Dive into How Hibernate Works

JPA vs Hibernate	Spec vs Implementation
Core Components	SessionFactory, Session, Transaction, Query
Spring Boot Auto-Config	application.properties settings
Session vs EntityManager	Equivalent API comparison
First-Level Cache	Session-scoped, always-on identity guarantee
SQL Generation	How @Entity maps to SQL

1 JPA vs Hibernate

Specification vs Implementation

JPA (Java Persistence API) is a specification -- a set of interfaces and rules defined by the Java EE / Jakarta EE standard. It tells you WHAT to do (e.g., use @Entity, use EntityManager) but does not provide the code that actually runs. Hibernate is the most popular JPA implementation -- it provides the concrete classes that execute SQL, manage caches, and handle transactions. Think of JPA as a blueprint and Hibernate as the finished building.

Java Example -- using JPA interfaces only (Hibernate runs underneath):

```
import javax.persistence.*;

@Entity
@Table(name = "employee")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    // getters and setters
}
```

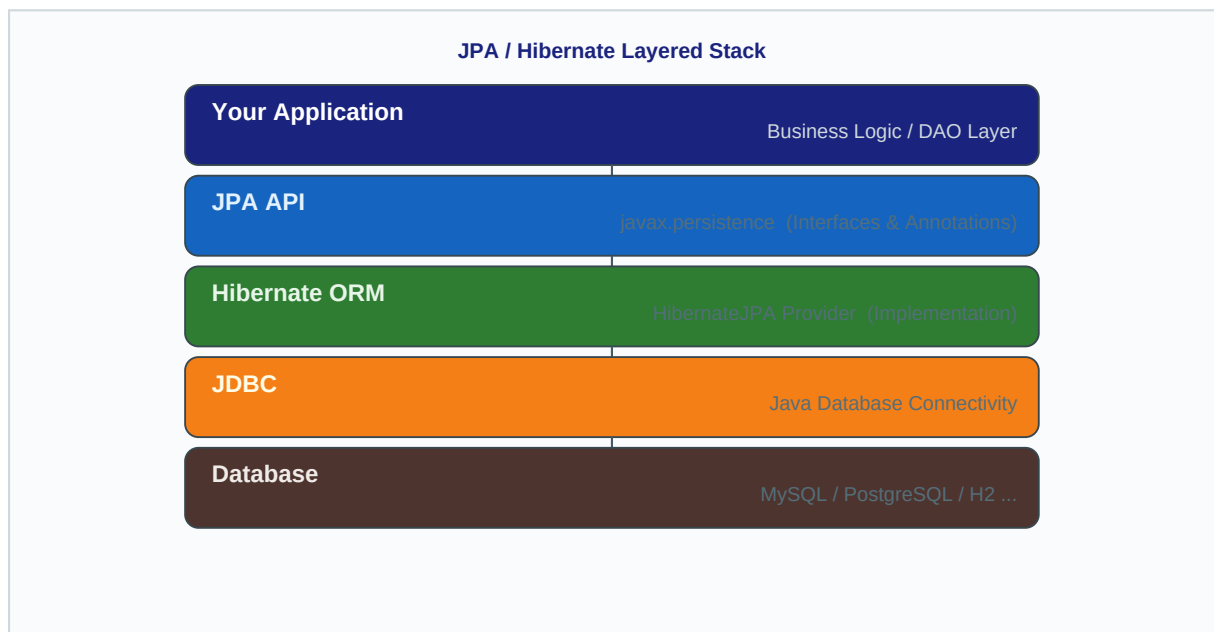


Figure 1 -- Layered stack: your code calls JPA interfaces; Hibernate implements them via JDBC.

Key Rules

- Always code to JPA interfaces (EntityManager, Query) -- never import Hibernate classes in business code.
- JPA is the contract; swapping Hibernate for EclipseLink should require zero business-code changes.
- Hibernate adds extras (HQL, Criteria API, 2nd-level cache) on top of the JPA standard.
- The @Entity annotation is JPA; Hibernate honours it automatically.

Q1. What is JPA?

JPA is a Java specification (a set of interfaces) that defines how Java objects are persisted in a relational database. It is not a library you run -- it needs an implementation.

Q2. What is Hibernate?

Hibernate is an ORM framework that implements the JPA specification. It translates Java object operations into SQL statements and manages the database connection lifecycle.

Q3. Can I use Hibernate without JPA?

Yes. Hibernate has its own native API (Session, SessionFactory). But using JPA interfaces is preferred because it keeps your code portable.

Q4. What package contains JPA annotations?

javax.persistence (Java EE) or jakarta.persistence (Jakarta EE 9+). Hibernate provides both.

Q5. Name one advantage of coding to JPA interfaces.

You can swap the JPA provider (e.g., replace Hibernate with EclipseLink) without rewriting business code, because all calls go through the standard JPA API.

2 Core Components

SessionFactory, Session, Transaction, Query

Hibernate has four main building blocks you will use every day:

SessionFactory -- Created once per application. Expensive to build. Thread-safe. Reads all @Entity mappings and prepares SQL templates.

Session -- Created per request or per unit of work. Lightweight. NOT thread-safe. Represents a single conversation with the database.

Transaction -- Wraps a unit of work. Call beginTransaction(), do your work, then commit() or rollback().

Query -- Lets you run JPQL / HQL or native SQL to retrieve data.

Java Example -- opening a Session and saving an entity:

```
SessionFactory sf = new Configuration()
    .configure("hibernate.cfg.xml")
    .addAnnotatedClass(Employee.class)
    .buildSessionFactory();

Session session = sf.openSession();
Transaction tx = session.beginTransaction();

Employee emp = new Employee();
emp.setName("Alice");
session.save(emp);           // INSERT is queued

tx.commit();                 // INSERT fires here
session.close();
```

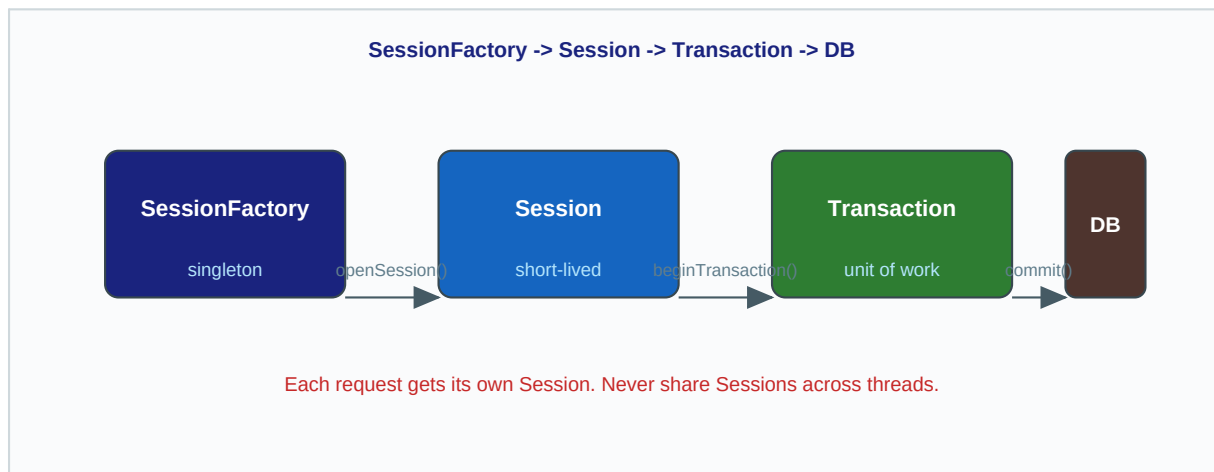


Figure 2 -- SessionFactory produces Sessions; each Session owns one Transaction.

Key Rules

- Build SessionFactory once at startup -- it is very expensive to build.
- Always close() a Session after use to release the database connection.
- Wrap every write operation in a Transaction; reads also benefit from one.
- Never share a Session between threads -- use one Session per thread/request.

Q1. What is the role of SessionFactory?

It is the root object of Hibernate. Created once, it holds the configuration, SQL caches, and connection pool. Use it to open individual Sessions.

Q2. Why is Session not thread-safe?

Session keeps a first-level cache and a change-tracker (dirty checking) that are tied to one conversation. Sharing it across threads would corrupt that state.

Q3. What happens if you forget tx.commit()?

The transaction is rolled back when the Session closes, and no changes reach the database.

Q4. What is JPQL?

Java Persistence Query Language -- an SQL-like language that operates on entity class names and field names instead of table and column names.

Q5. How do you run a JPQL query?

```
session.createQuery("FROM Employee WHERE name = :n", Employee.class).setParameter("n",  
"Alice").list();
```

3 Spring Boot Auto-Config

application.properties Hibernate Settings

When you add `spring-boot-starter-data-jpa` to your Maven/Gradle project, Spring Boot automatically creates the `DataSource`, `EntityManagerFactory` (which wraps `SessionFactory`), and `TransactionManager`. You configure Hibernate behaviour through `application.properties` -- no XML needed.

Key application.properties entries:

```
# Database connection
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret

# Hibernate dialect -- tells Hibernate which SQL flavor to generate
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

# DDL auto: none | validate | update | create | create-drop
spring.jpa.hibernate.ddl-auto=update

# Log every SQL statement Hibernate sends
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Minimal Spring Boot Repository Example:

```
@Repository
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {
    // Spring Data generates: SELECT * FROM employee WHERE name = ?
    List<Employee> findByName(String name);
}
```

Key Rules

- `spring.jpa.hibernate.ddl-auto=update` is fine for development; use `validate` or `none` in production.
- `show-sql=true` prints raw SQL to the console -- useful for debugging, noisy in production.
- The `dialect` property is optional from Hibernate 6 -- it auto-detects from the JDBC URL.
- `spring-boot-starter-data-jpa` brings in Hibernate, Spring Data JPA, and HikariCP in one dependency.

Q1. What does `ddl-auto=create-drop` do?

Creates the schema when the app starts and drops it when the app shuts down. Useful for integration tests but destroys production data.

Q2. How does Spring Boot know which entities to manage?

It scans all packages at and below the main `@SpringBootApplication` class for `@Entity` classes.

Q3. Where does Hibernate learn the DB dialect?

From `spring.jpa.properties.hibernate.dialect`, or auto-detected from the JDBC driver in Hibernate 6+.

Q4. Do you need `hibernate.cfg.xml` with Spring Boot?

No. Spring Boot replaces the XML config entirely with `application.properties` and auto-configuration.

Q5. What dependency adds Spring Data JPA and Hibernate?

`spring-boot-starter-data-jpa` in `pom.xml` or `build.gradle`.

4 Session vs EntityManager

Native Hibernate API vs JPA Standard API

Hibernate's native API uses Session. The JPA standard uses EntityManager. Hibernate's Session actually implements EntityManager, so they are the same object -- just accessed through different interfaces. When you use Spring Data JPA, you almost never touch either directly; but knowing both helps when you drop to custom queries.

Side-by-side comparison:

Operation	Session (Hibernate)	EntityManager (JPA)
Save new entity	session.save(obj)	em.persist(obj)
Load by primary key	session.get(Cls, id)	em.find(Cls, id)
Update detached	session.update(obj)	em.merge(obj)
Delete entity	session.delete(obj)	em.remove(obj)
Flush to DB	session.flush()	em.flush()
JPQL query	session.createQuery(hql)	em.createQuery(jpql)

Java Example -- using EntityManager in a Spring service:

```
@Service
public class EmployeeService {
    @PersistenceContext
    private EntityManager em;

    @Transactional
    public Employee findById(Long id) {
        return em.find(Employee.class, id);
    }

    @Transactional
    public void save(Employee e) {
        em.persist(e); // INSERT on commit
    }
}
```

Key Rules

- Prefer EntityManager in Spring apps -- it keeps code JPA-standard and portable.
- @PersistenceContext injects a thread-safe proxy; the real Session is per-transaction.
- session.save() returns the generated ID; em.persist() is void -- check em.find() afterwards.
- Both APIs do dirty checking: modify a managed entity and Hibernate generates UPDATE automatically.

Q1. Is EntityManager the same object as Session?

Yes. In Hibernate, Session extends EntityManager. You can cast: (Session) em.

Q2. When would you use Session instead of EntityManager?

When you need Hibernate-specific features like session.saveOrUpdate(), native SQL with addEntity(), or the Criteria API older style.

Q3. What does @Transactional do?

Spring opens a transaction before the method and commits (or rolls back on exception) after it. EntityManager/Session is bound to that transaction automatically.

Q4. What is a detached entity?

An entity that was once managed (inside a Session) but the Session was closed. Use merge() to re-attach it.

Q5. What is a managed entity?

An entity currently tracked by an open Session/EntityManager. Any field change is auto-flushed as an UPDATE when the transaction commits.

5 First-Level Cache

Session-Scoped, Always On, Identity Guarantee

Every Hibernate Session maintains a first-level cache (also called the persistence context). It is a simple Map from entity class + primary key to the entity object. When you load the same entity twice within one Session, Hibernate returns the same Java object from cache -- no second SQL query is issued. This cache is always on; you cannot disable it. It is automatically cleared when the Session closes.

Java Example -- seeing the identity guarantee:

```
Session session = sf.openSession();

Employee e1 = session.get(Employee.class, 1L); // SELECT fires
Employee e2 = session.get(Employee.class, 1L); // NO SQL -- returns cache hit

System.out.println(e1 == e2); // true -- same object reference

session.close(); // cache is discarded
```

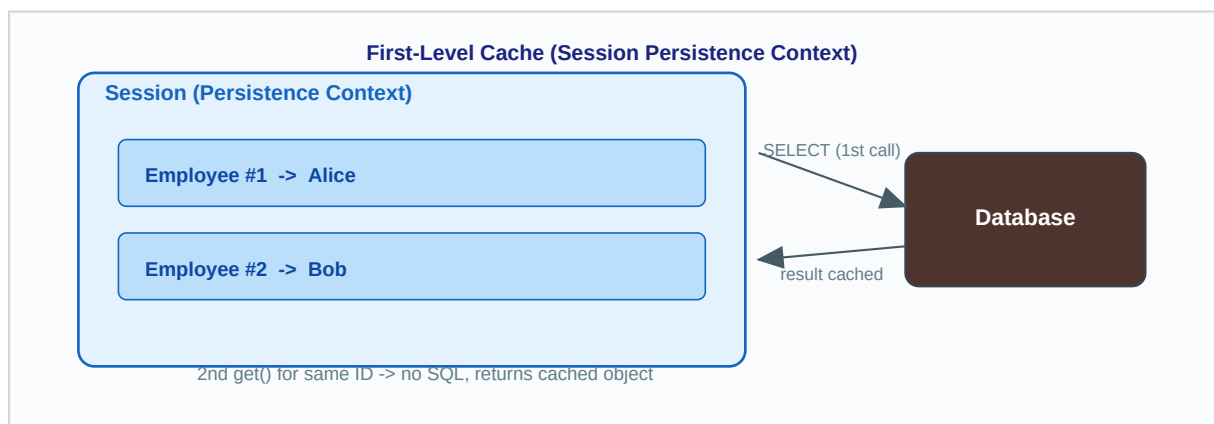


Figure 3 -- First-level cache inside a Session. Only the first get() hits the DB.

Key Rules

- The first-level cache is always active -- there is no configuration to turn it off.
- Cache scope is the Session. A new Session starts with an empty cache.
- Two Sessions loading the same row return DIFFERENT Java objects (no cross-session cache by default).
- Call `session.evict(entity)` to manually remove one entry, or `session.clear()` to empty the whole cache.
- The second-level cache (optional) is shared across Sessions and requires explicit setup.

Q1. What is the first-level cache?

A per-Session in-memory map that stores entities by (type, id). Prevents duplicate SQL for the same entity within one unit of work.

Q2. Is the first-level cache enabled by default?

Yes, always. You cannot disable it. You can only clear or evict individual entries.

Q3. What is the identity guarantee?

Within one Session, `session.get(Employee.class, 1L)` always returns the exact same Java object reference, no matter how many times you call it.

Q4. How do you force a fresh SELECT ignoring the cache?

Call `session.refresh(entity)` -- Hibernate re-reads the row from the DB and updates the cached object.

Q5. What happens to the cache when a Session closes?

It is discarded. The next Session starts with an empty persistence context.

6 SQL Generation

How Hibernate Maps @Entity to SQL

When Hibernate starts, it reads every @Entity class and builds internal SQL templates for INSERT, SELECT, UPDATE, and DELETE. These are stored in the SessionFactory. At runtime, Hibernate fills in the ? placeholders with actual values. You never write this SQL -- Hibernate generates it from your annotations.

Java Entity and the SQL Hibernate generates:

```
@Entity
@Table(name = "product")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "product_id")
    private Long id;

    @Column(name = "product_name", nullable = false)
    private String name;

    @Column(name = "price")
    private double price;
}
// Hibernate generates:
// INSERT INTO product (product_name, price) VALUES (?, ?)
// SELECT * FROM product WHERE product_id = ?
// UPDATE product SET product_name=?, price=? WHERE product_id=?
// DELETE FROM product WHERE product_id = ?
```

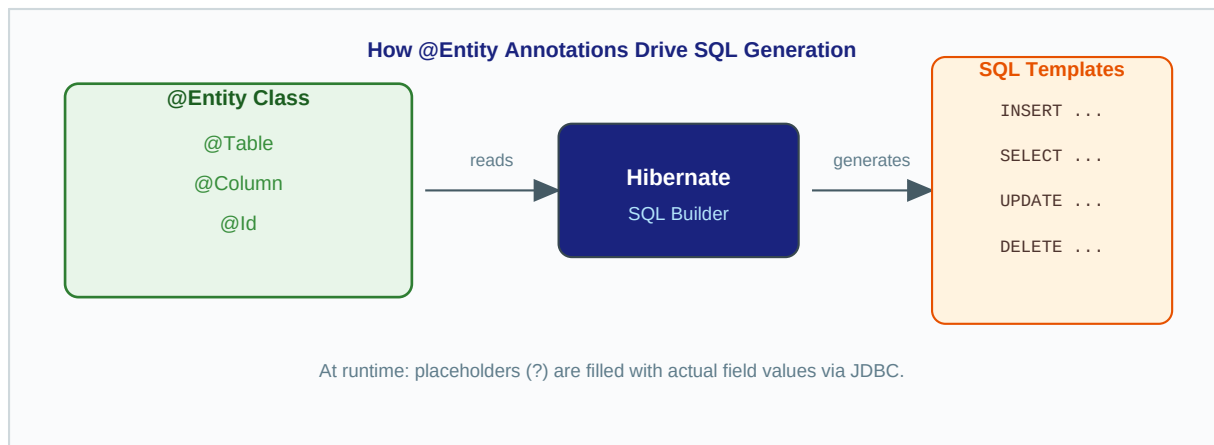


Figure 4 -- Hibernate reads annotations at startup and pre-builds SQL templates.

Key Rules

- @Table(name=...) overrides the default table name (which equals the class name).
- @Column(name=...) overrides the default column name (which equals the field name).
- nullable=false adds a NOT NULL constraint when Hibernate generates the DDL.
- Hibernate uses PreparedStatement internally -- all ? placeholders prevent SQL injection.
- show-sql=true in application.properties prints each generated SQL to the console.

Q1. When does Hibernate build the SQL templates?

At startup, when the SessionFactory (or EntityManagerFactory) is created. This is why startup is slow but runtime queries are fast.

Q2. What table name does Hibernate use if you omit @Table?

The class name. So 'class Product' maps to table 'Product' (case may vary by dialect).

Q3. What column name does Hibernate use if you omit @Column?

The field name. So 'private String productName' maps to column 'productName'.

Q4. Does Hibernate protect against SQL injection?

Yes. It uses JDBC PreparedStatements with ? placeholders, so user input is never concatenated into the SQL string.

Q5. What annotation marks the primary key?

@Id. Combined with @GeneratedValue, Hibernate also auto-generates the ID value using the database's auto-increment or a sequence.